

One-to-One Relationship



sql Relationship One to One explain all concept

SQL One-to-One Relationship (1:1) – Complete Concept

A **One-to-One Relationship** means:

| One row in Table A is connected to only one row in Table B.

And

| One row in Table B is connected to only one row in Table A.

Real Life Examples

Table A	Table B	Relationship
Person	Passport	One person has one passport
User	UserProfile	One user has one profile
Employee	ParkingSpace	One employee gets one parking space
Student	StudentIDCard	One student has one ID card

Example Diagram

Here:

- User 1 → only one profile
- Profile 1 → belongs to only one user

This is **One-to-One**.

How SQL Creates One-to-One Relationship

We use:

1. **Primary Key**
2. **Foreign Key**
3. **UNIQUE Constraint**

Important Terms

1. Primary Key

Uniquely identifies each row.

2. Foreign Key

Connects one table to another.

3. UNIQUE Constraint

Makes sure one value appears only once.

This is the MOST IMPORTANT part for One-to-One.

Method 1: Separate Primary Key + Unique Foreign Key

Create Parent Table

Create Child Table

Why UNIQUE is Important?

Without **UNIQUE**, many profiles could use same **user_id**.

That would become:

✗ One-to-Many

Example:

So we add:

Now only one profile per user is allowed.

Insert Data

Insert Users

Insert Profiles

Invalid Insert Example

✗ ERROR

Because:

already exists.

UNIQUE constraint prevents duplicate relationship.

Method 2: Shared Primary Key Relationship

This is a stronger One-to-One relationship.

Both tables share same primary key.

Example

Parent Table

Child Table

How It Works

Same ID used in both tables.

Example:

This guarantees exactly one matching row.

Difference Between Both Methods

Method	Description
UNIQUE Foreign Key	Most common
Shared Primary Key	Stronger relationship

Fetch Data Using JOIN

INNER JOIN

Output

DELETE Behavior

Suppose:

What happens to profile?

Depends on:

ON DELETE CASCADE

Automatically deletes child row.

Example

Delete user:

Then profile also deletes automatically.

ON UPDATE CASCADE

If parent ID changes, child updates automatically.

Full Example with Cascade

Advantages of One-to-One Relationship

Advantage	Explanation
Better organization	Separate sensitive data
Security	Store private data separately
Flexibility	Optional information
Faster queries	Smaller tables

When to Use One-to-One

Use when:

✓ Data is optional ✓ Data is sensitive ✓ Large data should be separated ✓ Logical separation needed

Example: User + Address

One user → One address

One-to-One vs One-to-Many

Feature	One-to-One	One-to-Many
Child rows	Only 1	Multiple
UNIQUE needed?	Yes	No
Example	User → Profile	User → Orders

One-to-One vs Many-to-Many

Relationship	Example
One-to-One	Person → Passport
One-to-Many	Customer → Orders
Many-to-Many	Students ↔ Courses

Common Interview Questions

Q1: How do you create One-to-One relationship?

Using:

- Foreign Key
- UNIQUE constraint

Q2: Why UNIQUE is required?

To prevent multiple child rows for same parent.

Q3: Difference between One-to-One and One-to-Many?

One-to-One allows only one related row.

One-to-Many allows many related rows.

Best Practice

Most developers use:

because it is flexible and easy.

Final Complete Example

Summary

Created using:

- PRIMARY KEY
- FOREIGN KEY

- UNIQUE

Short Formula

One-to-Few Relationship Using `populate()` in MongoDB + Mongoose

In MongoDB/Mongoose, a **One-to-Few** relationship means:

Usually implemented using:

- `ObjectId`
- `ref`
- `populate()`

Real Example

User → Few Phone Numbers

One user can have:

- Personal number
- Office number
- Emergency number

Only a few phone numbers.

How `populate()` Works

`populate()` replaces referenced IDs with actual document data.

Step 1: Create Phone Model

`phone.js`

Step 2: Create User Model

`user.js`

Important Part

This means:

Step 3: Insert Data

Create Phone Documents

Step 4: Create User

Database Storage

User Document

Only IDs are stored.

Step 5: Use `populate()`

Output

What `populate()` Did

Before populate:

After populate:

Visual Flow

One-to-Few Structure

This is implemented using:

Limiting to Few Records

Suppose:

Before adding:

Example

Add New Phone

Prevent 4th Phone

Complete Example

User Schema

Populate Query

Why Use `populate()`?

Benefit	Explanation
Cleaner database	Store references only
Faster updates	Separate documents
Reusable data	Same document reusable
Easy JOIN-like feature	Similar to SQL JOIN

`populate()` vs SQL JOIN

SQL	MongoDB
JOIN	<code>populate()</code>
Foreign Key	<code>ObjectId</code>
Table Relation	Document Reference

SQL Equivalent

MongoDB equivalent:

Real-World Examples

Parent	Few Child Docs
--------	----------------

User	Addresses
------	-----------

Product	Images
---------	--------

Student	Emergency Contacts
---------	--------------------

Profile	Social Links
---------	--------------

Important Interview Point

If interviewer asks:

| “How do you implement One-to-Few in Mongoose?”

Answer:

Final Summary

One-to-Few in Mongoose

Implemented using:

- `ObjectId`
- `ref`
- `populate()`

Formula

Main Syntax

Reference

Populate

one to squillions (in Details)

One-to-Squillions Relationship (Detailed Explanation)

What is One-to-Squillions?

“One-to-Squillions” is a database design term mostly used in:

- MongoDB
- NoSQL systems

It means:

The child records can become:

- Thousands
- Millions
- Billions

Meaning of “Squillions”

“Squillions” is not a real number.

It informally means:

Real-World Examples

Parent	Massive Child Records
User	Logs
Post	Comments
Video	Views
Website	Analytics events
IoT Device	Sensor readings
Customer	Transactions

Example

Video → Views

One video can have:

Wrong Design (Embedding)

Suppose you store all views inside video document.

Problem

This becomes VERY BAD because:

✗ Huge document size ✗ Slow queries ✗ Memory issues ✗ Update problems ✗
MongoDB document size limit (16MB)

Correct Design

Store child documents separately.

Structure

Example Collections

videos Collection

views Collection

Relationship

Why Separate Collection?

Because child records grow infinitely.

One-to-Squillions in MongoDB

MongoDB best practice:

✓ Parent document stays small ✓ Child documents stored separately ✓ Use references

Mongoose Implementation

Step 1: Create Parent Model

video.js

Step 2: Create Child Model

view.js

Important Design

Notice:

This creates:

Insert Example

Create Video

Add Millions of Views

Each view becomes separate document.

Why This Is Best?

Because:

without affecting parent document size.

Query Example

Find Views of Video

Count Total Views

Visual Diagram

Difference from One-to-Few

Feature	One-to-Few	One-to-Squillions
Child count	Small	Massive
Storage	Array possible	Separate collection
Embedding	Good	Bad
Performance	Easy	Needs optimization

Embedding vs Referencing

One-to-Few

Embedding often works:

One-to-Squillions

Embedding becomes dangerous:

 Very bad design.

Best Practice

For One-to-Squillions:

 Use separate collection  Use references  Add indexes  Use pagination  Use aggregation

Indexing Important

Because millions of records exist.

Add index:

Makes queries fast.

Pagination Example

Never load millions at once.

Use:

Common Use Cases

Parent	Squillions Child Docs
Post	Comments
User	Activity Logs
Product	Reviews
Server	Logs
Device	Sensor Data

SQL Equivalent

In SQL:

but with:

- Huge scaling
- Performance optimization
- Partitioning
- Indexing

Important MongoDB Principle

MongoDB rule:

Very important interview point.

Interview Question

Q1: What is One-to-Squillions?

Answer:

Q2: Why avoid embedding?

Because:

- Huge document size
- Performance issues
- MongoDB 16MB limit

Q3: Best implementation?

Use:

Final Summary

One-to-Squillions

Best Design

Formula

Main Rule

Reading Documentation (Docs) — Complete Guide for Beginners

Documentation (“Docs”) is the official guide that explains:

- How a technology works
- Installation steps
- Syntax
- Methods/functions
- Examples
- Best practices

Learning to read docs is one of the most important developer skills.

What Are Docs?

Docs are like a:

Examples:

- React Docs
- MongoDB Docs
- Node.js Docs
- Express Docs
- Mongoose Docs

Why Reading Docs Is Important

Most professional developers learn by:

✅ Reading docs ✅ Testing examples ✅ Experimenting

Not by memorizing tutorials.

Benefits of Reading Docs

Benefit	Explanation
Latest Information	Docs stay updated
Accurate Syntax	Official source
Better Understanding	Learn concepts deeply
Debugging Help	Find solutions quickly
Interview Preparation	Understand real concepts

Example Documentation Websites

React

[React Docs](#)

MongoDB

[MongoDB Docs](#)

Mongoose

[Mongoose Docs](#)

Node.js

[Node.js Docs](#)

Express.js

[Express Docs](#)

How to Read Docs Properly

Most beginners make mistake:

✗ Reading every line ✗ Memorizing syntax only

Correct method:

✓ Learn structure ✓ Understand examples ✓ Practice immediately

Step-by-Step Method

Step 1: Understand the Purpose

Ask:

Example:

- Express → Backend framework
- Mongoose → MongoDB ODM
- React → UI library

Step 2: Read Installation First

Almost every docs page starts with installation.

Example:

This tells you:

- package name
- how to setup

Step 3: Read Basic Example

Docs usually provide a simple example.

Example from Mongoose:

Do NOT just copy.

Understand:

- What connect() does
- Why URL is needed
- What database name means

Step 4: Understand Main Concepts

Example:

In Mongoose

Main concepts are:

- Schema
- Model
- Document
- Validation
- Populate

Step 5: Read Syntax Carefully

Example:

Ask:

Step 6: Test Everything Yourself

Reading alone is not enough.

Always:

Example

After reading:

Try:

Experimenting improves understanding.

How Professional Developers Read Docs

Professionals do NOT read full docs.

They:

✓ Search exact problem ✓ Read examples ✓ Check syntax ✓ Test quickly

Smart Way to Read Docs

Suppose you need:

Search:

Read only relevant section.

Important Parts of Documentation

Section	Purpose
Introduction	What tool does
Installation	Setup
Quick Start	Basic usage
API Reference	Methods/functions
Examples	Real code
Guides/Tutorials	Detailed explanation

Understanding API Reference

Example:

Docs explain:

- Parameters
- Return type
- Examples

- Options

Example Breakdown

Part	Meaning
User	Model
find()	Query method
{ age: 18 }	Filter

Reading Errors in Docs

Docs also explain:

- Errors
- Warnings
- Edge cases

Very important for debugging.

Common Beginner Mistakes

Mistake 1

Trying to memorize everything.

 Wrong

Mistake 2

Skipping examples.

 Wrong

Mistake 3

Copy-paste without understanding.

 Wrong

Mistake 4

Fear of large documentation.

 Wrong

Even seniors search docs daily.

Best Strategy

Learn in Layers

First Read

Understand overview.

Second Read

Understand syntax.

Third Read

Build project using it.

Example: Reading Mongoose Docs

Learn Order

1. Installation

2. Connection

3. Schema

4. Model

5. CRUD

6. Populate

How to Practice Reading Docs

Method

1. Pick one topic
2. Read docs for 20 mins
3. Write code yourself
4. Break code intentionally
5. Fix using docs

This builds real skill.

Golden Rule

Reading Docs Efficiently

Use:

Ctrl + F

Search keywords quickly.

Example:

Use Official Docs Only

Prefer official documentation over random blogs.

Because:

✅ Updated ✅ Accurate ✅ Trusted

Docs vs Tutorials

Tutorials	Docs
Step-by-step	Reference-based
Beginner friendly	Detailed
Slower learning	Faster growth
Guided	Self-learning

Best approach:

Real Developer Workflow

Interview Tip

If interviewer asks:

| “Do developers memorize everything?”

Best answer:

Final Summary

Documentation Helps You

✓ Learn technology ✓ Understand syntax ✓ Solve errors ✓ Build projects ✓ Become independent developer

Best Formula

Most Important Skill
